LifeLink

Blood Bank & Donor Finder System

C Mini-Project — Source Code and Output Screenshots

Name	M.V. VAMSI KRISHNA
Register number	25BEV7052
Course code	CSE 1022
Course / Slot	Introduction to Programming, F2+TF2
Faculty	Dr A. Manimuthu
Institution	VIT-AP UNIVERSITY

Project overview

In a medical emergency, finding a donor who is blood-compatible with the patient and currently eligible to donate takes too long when records are on paper or in chat groups. **LifeLink** is a menu-driven C program that registers donors the way government portals do, applies the national donor-selection rules automatically, and lists eligible compatible donors in seconds.

Main features

- **Registration:** name as on Aadhaar, father's/spouse's name, date of birth, gender, blood group, weight, mobile, email, full address (street, city, district, state/UT, PIN), Aadhaar number with Verhoeff checksum and consent, donation history, health screening and a declaration.
- **Emergency finder:** compatible blood groups, units in stock and eligible donors for a patient, exact matches first.
- **Donation recording** with the eligibility check and a 7-question pre-donation screening.
- Search by blood group, city, district or ID; blood stock with low-stock alerts and safe substitutes; statistics; text report.

Eligibility rules — NBTC Guidelines for Blood Donor Selection, 2017 (Govt. of India)

- Age 18–65 years (a first-time donor up to 60); body weight at least 45 kg.
- At least **90 days (men)** or **120 days (women)** between whole-blood donations.
- Health screening questions as on the government's e-RaktKosh portal.

Syllabus concepts used (Modules 1–4)

Control structures; arrays, 2-D arrays (Aadhaar checksum) and character arrays with our own string functions; functions, call by value and by reference, recursion (binary search); pointers and pointer arithmetic, `malloc` / `calloc` / `realloc` / `free`, a pointer to a compare function used by our own sort; structures, an array of structures, a union (search key); `fopen` / `fread` / `fwrite` / `fseek` / `fprintf` with a binary data file. Only `<stdio.h>` and `<stdlib.h>` are used — nothing outside the syllabus.

How to compile and run

```
gcc bloodbank.c -o bloodbank
./bloodbank
```

Compiles with `gcc -std=c99 -Wall -Wextra -pedantic` with zero warnings; all 22 test cases passed.

Links

Web prototype: lifelinkbloodbank.vercel.app
Source on GitHub: github.com/krishmanikanta31-pixel/LifeLink-Blood-Bank

Source code — bloodbank.c

```c
/*
 * ================================================================
 *  LifeLink - Blood Bank & Donor Finder System
 * ----------------------------------------------------------------
 *  Problem statement:
 *    In a medical emergency, finding a donor whose blood group is
 *    compatible with the patient AND who is currently eligible to
 *    donate wastes precious time when records are kept on paper or
 *    in chat groups. LifeLink registers donors the way government
 *    portals do (personal details, address, Aadhaar, donation
 *    history, health screening), applies the national donor
 *    selection rules, finds eligible compatible donors instantly,
 *    and warns the blood bank when any blood group is running low.
 *
 *  Eligibility rules - NBTC Guidelines for Blood Donor Selection
 *  (2017), Govt. of India:
 *    - age 18 to 65 years; a first-time donor must not be above 60
 *    - body weight at least 45 kg
 *    - whole-blood donation gap: 90 days for men, 120 days for women
 *    - health screening questions as on the e-RaktKosh portal
 *
 *  Syllabus concepts used (Modules 1 - 4):
 *    M1: constants, variables, operators, expressions, selection
 *        (if / switch), branching and looping statements, I/O.
 *    M2: arrays (incl. 2-D arrays for the Verhoeff checksum and
 *        character arrays with our own string functions), function
 *        prototypes / definitions / calls, call by value and by
 *        reference, scope of variables, recursion (binary search).
 *    M3: pointers, pointer arithmetic, malloc / calloc / realloc /
 *        free, pointer to functions (own sort with a compare
 *        function), pointers and arrays.
 *    M4: structures (incl. an array of structures for the screening
 *        questions), union (search key), file operations fopen /
 *        fread / fwrite / fseek / fprintf, binary file read & write.
 *  Only <stdio.h> and <stdlib.h> are used; no topic outside the
 *  syllabus. Today's date is entered when the program starts.
 *
 *  Name   : M.V. VAMSI KRISHNA
 *  Reg No : 25BEV7052
 *  Course : Introduction to Programming (Slot F2+TF2)
 * ================================================================
 */

#include <stdio.h>
#include <stdlib.h>

#define DATA_FILE      "lifelink.dat"
#define REPORT_FILE    "donor_report.txt"
#define NUM_GROUPS     8
#define NUM_STATES     36
#define LOW_STOCK      5     /* alert when units fall below this */

/* NBTC 2017 donor selection rules */
#define MIN_AGE        18
#define MAX_AGE        65
#define MAX_FIRST_AGE  60    /* a first-time donor must not be above 60 */
#define MIN_WEIGHT     45    /* kg */
#define GAP_MEN        90    /* days between whole-blood donations */
#define GAP_WOMEN      120   /* also used for donors who identify as Other */

typedef struct {
    int d, m, y;               /* y == 0 means "never" */
} Date;

typedef struct {
    int  id;
    char name[40];             /* as on Aadhaar */
    char guardian[40];         /* father's / spouse's name */
    Date dob;
    char gender;               /* M / F / O */
    char group[4];             /* e.g. "AB+" */
    int  weight;               /* kg */
    char phone[11];            /* 10 digits */
    char email[50];            /* optional, "" when not given */
    char address[60];          /* house no. / street / locality */
    char city[30];
    char district[30];
    char state[40];
    char pin[7];               /* 6 digits */
    char aadhaarLast4[5];      /* only the last 4 digits are stored */
    unsigned long aadhaarHash; /* fingerprint of the full number, to block duplicates */
    Date lastDonation;
    int  totalDonations;
} Donor;

typedef struct {
    Donor *list;               /* dynamic array, always sorted by id */
    int count;
    int capacity;
    int nextId;
} Registry;
```

```
/* a health screening question, worded as on the e-RaktKosh portal */
typedef struct {
    const char *text;
    char deferIf;              /* 'y' or 'n': the answer that needs a deferral */
    int  atRegistration;       /* 1 = also asked when registering */
    const char *reason;
} Question;

/* search modes */
#define BY_GROUP    1
#define BY_CITY     2
#define BY_DISTRICT 3
#define BY_ID       4

/*
 * A search uses only ONE key at a time (a group, a place name or an
 * id), so the keys share the same memory in a union, not a struct.
 */
typedef union {
    int  id;
    char group[4];
    char place[30];            /* city or district */
} SearchKey;

/* file layout: version | count | nextId | stock[8] | Donor records */
#define FILE_VERSION 3
#define HEADER_SIZE ((long)((3 + NUM_GROUPS) * sizeof(int)))

const char *GROUPS[NUM_GROUPS] = {"A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"};

const char *STATES[NUM_STATES] = {
    "Andhra Pradesh", "Arunachal Pradesh", "Assam", "Bihar", "Chhattisgarh", "Goa", "Gujarat",
    "Haryana", "Himachal Pradesh", "Jharkhand", "Karnataka", "Kerala", "Madhya Pradesh",
    "Maharashtra", "Manipur", "Meghalaya", "Mizoram", "Nagaland", "Odisha", "Punjab",
    "Rajasthan", "Sikkim", "Tamil Nadu", "Telangana", "Tripura", "Uttar Pradesh", "Uttarakhand",
    "West Bengal", "Andaman and Nicobar Islands", "Chandigarh",
    "Dadra and Nagar Haveli and Daman and Diu", "Delhi", "Jammu and Kashmir", "Ladakh",
    "Lakshadweep", "Puducherry"
};

#define NUM_QUESTIONS 7
const Question QUESTIONS[NUM_QUESTIONS] = {
    {"Are you currently taking any medication (e.g., antibiotics, blood thinners)?", 'y', 1, "on medication"},
    {"Have you had any infection, fever, cold, cough, weakness, dizziness, or fatigue today?", 'y', 1, "unwell"},
    {"Have you undergone any surgery or major dental procedure recently (last 6-12 months)?", 'y', 1, "recent surgery"},
    {"Have you had any tattoos, piercings, or acupuncture in the last 6 months?", 'y', 1, "tattoo / piercing in last 6
months"},
    {"Have you consumed alcohol in the last 24 hours?", 'y', 1, "alcohol in last 24 hours"},
    {"Did you have at least 6 hours of sleep last night?", 'n', 0, "less than 6 hours of sleep"},
    {"Did you eat a light (non-oily) meal 2-3 hours before donating?", 'n', 0, "no light meal before donating"}
};

/* Verhoeff checksum tables - UIDAI uses this for the last Aadhaar digit */
const int VERHOEFF_D[10][10] = {
    {0,1,2,3,4,5,6,7,8,9}, {1,2,3,4,0,6,7,8,9,5}, {2,3,4,0,1,7,8,9,5,6}, {3,4,0,1,2,8,9,5,6,7},
    {4,0,1,2,3,9,5,6,7,8}, {5,9,8,7,6,0,4,3,2,1}, {6,5,9,8,7,1,0,4,3,2}, {7,6,5,9,8,2,1,0,4,3},
    {8,7,6,5,9,3,2,1,0,4}, {9,8,7,6,5,4,3,2,1,0}
};
const int VERHOEFF_P[8][10] = {
    {0,1,2,3,4,5,6,7,8,9}, {1,5,7,6,2,8,3,0,9,4}, {5,8,0,3,7,9,6,1,4,2}, {8,9,1,6,0,4,3,5,2,7},
    {9,4,5,3,1,2,6,8,7,0}, {4,2,8,6,5,7,3,9,0,1}, {2,7,9,3,8,0,6,4,1,5}, {7,0,4,6,9,1,3,2,5,8}
};

int  stock[NUM_GROUPS];  /* units of blood available per group */
Date today;

/* ---------------------------------------------------------------------
 *  Own character and string functions (character arrays + pointers)
 * --------------------------------------------------------------------- */

int isDigitChar(char c)
{
    return c >= '0' && c <= '9';
}

int isLetter(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

int isSpaceChar(char c)
{
    return c == ' ' || c == '\t' || c == '\n' || c == '\r';
}

char lowerCase(char c)
{
    return (c >= 'A' && c <= 'Z') ? (char)(c - 'A' + 'a') : c;
}

char upperCase(char c)
{
    return (c >= 'a' && c <= 'z') ? (char)(c - 'a' + 'A') : c;
}
```

```
/* length of a string - pointer arithmetic: end pointer minus start pointer */
int strLength(const char *s)
{
    const char *p = s;

    while (*p != '\0')
        p++;
    return (int)(p - s);
}

void strCopy(char *dest, const char *src)
{
    while ((*dest++ = *src++) != '\0')
        ;
}

void strAppend(char *dest, const char *src)
{
    strCopy(dest + strLength(dest), src);
}

/* 0 when equal, otherwise the difference of the first characters that differ */
int strCompare(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}

/* pointer to the first (or last) c in s, or NULL when it is not there */
const char *strFind(const char *s, char c)
{
    for (; *s != '\0'; s++)
        if (*s == c)
            return s;
    return NULL;
}

const char *strFindLast(const char *s, char c)
{
    const char *last = NULL;

    for (; *s != '\0'; s++)
        if (*s == c)
            last = s;
    return last;
}

/* removes spaces at both ends of s */
void trim(char *s)
{
    int start = 0, i, n;

    while (s[start] != '\0' && isSpaceChar(s[start]))
        start++;
    for (i = 0; s[start + i] != '\0'; i++)      /* shift the text left */
        s[i] = s[start + i];
    s[i] = '\0';

    n = strLength(s);
    while (n > 0 && isSpaceChar(s[n - 1]))
        s[--n] = '\0';
}

/* case-insensitive compare */
int compareIgnoreCase(const char *a, const char *b)
{
    while (*a && *b) {
        int diff = lowerCase(*a) - lowerCase(*b);
        if (diff != 0)
            return diff;
        a++;
        b++;
    }
    return lowerCase(*a) - lowerCase(*b);
}

/* ------------------------------------------------------------------
 *  Safe input functions (fgets based, never leave junk in stdin)
 * ------------------------------------------------------------------ */

void readLine(const char *prompt, char *buf, int size)
{
    printf("%s", prompt);
    fflush(stdout);

    if (fgets(buf, size, stdin) == NULL) {
        printf("\nInput ended. Exiting.\n");
        exit(0);
    }
    if (strFind(buf, '\n') == NULL) {        /* line too long: discard the rest */
        int c;
```

```
        while ((c = getchar()) != '\n' && c != EOF)
            ;
    }
    trim(buf);
}

int readInt(const char *prompt, int min, int max)
{
    char buf[32], extra;
    int value;

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        /* exactly one number and nothing after it */
        if (sscanf(buf, "%d%c", &value, &extra) == 1 && value >= min && value <= max)
            return value;
        printf("  ! Please enter a number between %d and %d.\n", min, max);
    }
}

int readYesNo(const char *prompt)
{
    char buf[8];

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (buf[0] == 'y' || buf[0] == 'Y')
            return 1;
        if (buf[0] == 'n' || buf[0] == 'N')
            return 0;
        printf("  ! Please answer y or n.\n");
    }
}

/* letters, spaces and dots only (names, city, district) */
void readWords(const char *prompt, char *buf, int size)
{
    for (;;) {
        int ok = 1, i;

        readLine(prompt, buf, size);
        if (buf[0] == '\0')
            ok = 0;
        for (i = 0; buf[i] && ok; i++)
            if (!isLetter(buf[i]) && buf[i] != ' ' && buf[i] != '.')
                ok = 0;
        if (ok)
            return;
        printf("  ! Required. Use letters and spaces only.\n");
    }
}

void readPhone(const char *prompt, char *buf)
{
    char tmp[32];

    for (;;) {
        int ok, i;

        readLine(prompt, tmp, sizeof tmp);
        ok = strLength(tmp) == 10 && tmp[0] >= '6' && tmp[0] <= '9';
        for (i = 0; tmp[i] && ok; i++)
            if (!isDigitChar(tmp[i]))
                ok = 0;
        if (ok) {
            strCopy(buf, tmp);
            return;
        }
        printf("  ! Enter a valid 10-digit mobile number (starts with 6-9).\n");
    }
}

/* optional: empty is allowed, otherwise name@domain.tld */
void readEmail(const char *prompt, char *buf, int size)
{
    for (;;) {
        const char *at, *dot;

        readLine(prompt, buf, size);
        if (buf[0] == '\0')
            return;
        at = strFind(buf, '@');
        dot = at ? strFindLast(at, '.') : NULL;
        if (at && at != buf && strFind(at + 1, '@') == NULL && strFind(buf, ' ') == NULL &&
            dot && dot > at + 1 && strLength(dot) >= 3)
            return;
        printf("  ! Enter a valid email address, or press Enter to skip.\n");
    }
}

void readAddress(const char *prompt, char *buf, int size)
{
    for (;;) {
        readLine(prompt, buf, size);
        if (strLength(buf) >= 5)
```

```
            return;
        printf("  ! Enter the house number and street.\n");
    }
}

void readPin(const char *prompt, char *buf)
{
    char tmp[32];

    for (;;) {
        int ok, i;

        readLine(prompt, tmp, sizeof tmp);
        ok = strLength(tmp) == 6 && tmp[0] >= '1' && tmp[0] <= '9';
        for (i = 0; tmp[i] && ok; i++)
            if (!isDigitChar(tmp[i]))
                ok = 0;
        if (ok) {
            strCopy(buf, tmp);
            return;
        }
        printf("  ! PIN code must be 6 digits and cannot start with 0.\n");
    }
}

/* the state / UT must match one of the 36 official names (any letter case) */
void readState(const char *prompt, char *out)
{
    char buf[64];

    for (;;) {
        int i;

        readLine(prompt, buf, sizeof buf);
        for (i = 0; i < NUM_STATES; i++) {
            if (compareIgnoreCase(buf, STATES[i]) == 0) {
                strCopy(out, STATES[i]);
                return;
            }
        }
        printf("  ! Type the full name of a state or UT, e.g. Andhra Pradesh, Telangana, Delhi.\n");
    }
}

char readGender(const char *prompt)
{
    char buf[8];

    for (;;) {
        char g;

        readLine(prompt, buf, sizeof buf);
        g = upperCase(buf[0]);
        if (buf[1] == '\0' && (g == 'M' || g == 'F' || g == 'O'))
            return g;
        printf("  ! Enter M, F or O.\n");
    }
}

/* ---------------------------------------------------------------------
 *  Blood group logic
 * --------------------------------------------------------------------- */

/* returns index into GROUPS, or -1 if the text is not a blood group */
int groupIndex(const char *g)
{
    int i;

    for (i = 0; i < NUM_GROUPS; i++)
        if (compareIgnoreCase(g, GROUPS[i]) == 0)
            return i;
    return -1;
}

void readBloodGroup(const char *prompt, char *out)
{
    char buf[16];

    for (;;) {
        int idx;

        readLine(prompt, buf, sizeof buf);
        idx = groupIndex(buf);
        if (idx >= 0) {
            strCopy(out, GROUPS[idx]);   /* store in standard upper-case form */
            return;
        }
        printf("  ! Valid groups: A+ A- B+ B- AB+ AB- O+ O-\n");
    }
}

/*
 * Red-cell compatibility rule:
 *   - every antigen (A, B) the donor has, the recipient must also have
 *   - an Rh+ donor can only give to an Rh+ recipient
```

```
 */
int canDonate(const char *donor, const char *recipient)
{
    if (strFind(donor, 'A') && !strFind(recipient, 'A'))
        return 0;
    if (strFind(donor, 'B') && !strFind(recipient, 'B'))
        return 0;
    if (donor[strLength(donor) - 1] == '+' && recipient[strLength(recipient) - 1] == '-')
        return 0;
    return 1;
}

/* ------------------------------------------------------------------
 *  Aadhaar check
 * ------------------------------------------------------------------ */

/* Verhoeff checksum: the whole 12-digit number must give 0 */
int verhoeffValid(const char *digits)
{
    int c = 0, i, n = (int)strLength(digits);

    for (i = 0; i < n; i++)
        c = VERHOEFF_D[c][VERHOEFF_P[i % 8][digits[n - 1 - i] - '0']];
    return c == 0;
}

/* djb2 hash of the digits - lets us spot a duplicate without storing the number */
unsigned long aadhaarFingerprint(const char *digits)
{
    unsigned long h = 5381;

    while (*digits)
        h = h * 33 + (unsigned char)*digits++;
    return h;
}

/* reads "XXXX XXXX XXXX", checks it and returns the last 4 digits and the fingerprint */
void readAadhaar(const char *prompt, const Registry *r, char *last4, unsigned long *hash)
{
    char buf[40], digits[16];

    for (;;) {
        int i, n = 0, ok = 1;
        const Donor *d;

        readLine(prompt, buf, sizeof buf);
        for (i = 0; buf[i] && ok; i++) {
            if (isDigitChar(buf[i]) && n < 12)
                digits[n++] = buf[i];
            else if (buf[i] != ' ' && buf[i] != '-')
                ok = 0;
        }
        digits[n] = '\0';
        if (!ok || n != 12 || buf[i] != '\0' || digits[0] < '2') {
            printf("  ! Aadhaar must be 12 digits and cannot start with 0 or 1.\n");
            continue;
        }
        if (!verhoeffValid(digits)) {
            printf("  ! Invalid Aadhaar number (checksum failed). Please check the digits.\n");
            continue;
        }
        *hash = aadhaarFingerprint(digits);
        for (d = r->list; d < r->list + r->count; d++) {
            if (d->aadhaarHash == *hash)
                break;
        }
        if (d < r->list + r->count) {
            printf("  ! This Aadhaar is already registered to donor %d.\n", d->id);
            continue;
        }
        strCopy(last4, digits + 8);
        printf("  Aadhaar number is valid.\n");
        return;
    }
}

/* ------------------------------------------------------------------
 *  Date logic
 * ------------------------------------------------------------------ */

int isLeap(int y)
{
    return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
}

int daysInMonth(int m, int y)
{
    const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return (m == 2 && isLeap(y)) ? 29 : days[m - 1];
}

int isValidDate(Date dt)
{
    return dt.y >= 1900 && dt.y <= 2100 &&
           dt.m >= 1 && dt.m <= 12 &&
```

```
           dt.d >= 1 && dt.d <= daysInMonth(dt.m, dt.y);
}

/* converts a date to a day number so two dates can be subtracted */
long dateToDays(Date dt)
{
    int y = dt.y, m = dt.m;

    if (m < 3) {
        y--;
        m += 12;
    }
    return 365L * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + dt.d;
}

/* age in completed years on a given date */
int ageOn(Date dob, Date on)
{
    int age = on.y - dob.y;

    if (on.m < dob.m || (on.m == dob.m && on.d < dob.d))
        age--;
    return age;
}

/* today's date is entered once when the program starts */
Date readToday(void)
{
    char buf[32];
    Date dt;

    for (;;) {
        readLine("Enter today's date (dd/mm/yyyy): ", buf, sizeof buf);
        if (sscanf(buf, "%d/%d/%d", &dt.d, &dt.m, &dt.y) == 3 && isValidDate(dt) && dt.y >= 2024)
            return dt;
        printf("  ! Invalid date. Use dd/mm/yyyy, e.g. 25/10/2026.\n");
    }
}

void formatDate(Date dt, char *out)
{
    if (dt.y == 0)
        strCopy(out, "Never");
    else
        sprintf(out, "%02d/%02d/%04d", dt.d, dt.m, dt.y);
}

/* dd/mm/yyyy, not in the future; "0" means never when allowNever is set */
Date readDate(const char *prompt, int allowNever)
{
    char buf[32];
    Date dt;

    for (;;) {
        readLine(prompt, buf, sizeof buf);
        if (allowNever && strCompare(buf, "0") == 0) {
            dt.d = dt.m = dt.y = 0;
            return dt;
        }
        if (sscanf(buf, "%d/%d/%d", &dt.d, &dt.m, &dt.y) == 3 && isValidDate(dt)) {
            if (dateToDays(dt) <= dateToDays(today))
                return dt;
            printf("  ! Date cannot be in the future.\n");
        } else {
            printf("  ! Invalid date. Use dd/mm/yyyy%s.\n", allowNever ? ", or 0 if never donated" : "");
        }
    }
}

/* ------------------------------------------------------------------
 *  Eligibility (NBTC 2017) and health screening (e-RaktKosh)
 * ------------------------------------------------------------------ */

int gapDays(char gender)
{
    return gender == 'M' ? GAP_MEN : GAP_WOMEN;
}

const char *genderWord(char gender)
{
    return gender == 'M' ? "men" : gender == 'F' ? "women" : "donors who identify as Other";
}

/*
 * 0  = eligible now
 * >0 = days the donor still has to wait
 * -1 = not eligible (age or weight); *reason says why
 */
int checkEligibility(const Donor *d, const char **reason)
{
    int age = ageOn(d->dob, today);
    long since;

    *reason = "";
    if (age < MIN_AGE)          { *reason = "under 18";                 return -1; }
```

```
    if (age > MAX_AGE)          { *reason = "over 65";                  return -1; }
    if (d->weight < MIN_WEIGHT) { *reason = "weight below 45 kg";       return -1; }
    if (d->lastDonation.y == 0) {
        if (age > MAX_FIRST_AGE) { *reason = "first-time donor over 60"; return -1; }
        return 0;
    }
    since = dateToDays(today) - dateToDays(d->lastDonation);
    if (since >= gapDays(d->gender))
        return 0;
    *reason = d->gender == 'M' ? "90-day gap for men" : "120-day gap for women";
    return (int)(gapDays(d->gender) - since);
}

int isEligible(const Donor *d)
{
    const char *reason;
    return checkEligibility(d, &reason) == 0;
}

/*
 * Asks the screening questions (only the registration ones when
 * atRegistration is 1). Returns how many answers need a deferral and
 * lists their reasons in 'reasons'.
 */
int screening(int atRegistration, char *reasons, int size)
{
    int i, deferrals = 0;
    char prompt[160];

    reasons[0] = '\0';
    for (i = 0; i < NUM_QUESTIONS; i++) {
        const Question *q = &QUESTIONS[i];
        int yes;

        if (atRegistration && !q->atRegistration)
            continue;
        sprintf(prompt, "  %s (y/n): ", q->text);
        yes = readYesNo(prompt);
        if ((yes && q->deferIf == 'y') || (!yes && q->deferIf == 'n')) {
            if (deferrals > 0 && (int)strLength(reasons) + 2 < size)
                strAppend(reasons, ", ");
            if ((int)(strLength(reasons) + strLength(q->reason)) < size)
                strAppend(reasons, q->reason);
            deferrals++;
        }
    }
    return deferrals;
}

/* ------------------------------------------------------------------
 *  Registry (dynamic array) operations
 * ------------------------------------------------------------------ */

void ensureCapacity(Registry *r)
{
    Donor *bigger;
    int newCapacity;

    if (r->count < r->capacity)
        return;
    newCapacity = r->capacity ? r->capacity * 2 : 8;
    if (r->list == NULL)
        bigger = malloc(newCapacity * sizeof(Donor));             /* first block */
    else
        bigger = realloc(r->list, newCapacity * sizeof(Donor));   /* grow: 8, 16, 32 ... */
    if (bigger == NULL) {
        printf("Out of memory!\n");
        exit(1);
    }
    r->list = bigger;
    r->capacity = newCapacity;
}

/* recursive binary search - works because the list stays sorted by id */
int findById(const Donor *list, int low, int high, int id)
{
    int mid;

    if (low > high)
        return -1;
    mid = low + (high - low) / 2;
    if (list[mid].id == id)
        return mid;
    if (list[mid].id < id)
        return findById(list, mid + 1, high, id);
    return findById(list, low, mid - 1, id);
}

int askForDonor(const Registry *r)
{
    int id = readInt("Enter donor ID: ", 1, 999999);
    int idx = findById(r->list, 0, r->count - 1, id);

    if (idx < 0)
        printf("  ! No donor with ID %d.\n", id);
```

```
    return idx;
}

/* ------------------------------------------------------------------
 *  File handling
 *  Layout: count | nextId | stock[8] | Donor records
 * ------------------------------------------------------------------ */

int saveData(const Registry *r)
{
    FILE *fp = fopen(DATA_FILE, "wb");
    int version;

    if (fp == NULL) {
        printf("  ! Could not save data to %s\n", DATA_FILE);
        return 0;
    }
    version = FILE_VERSION;
    fwrite(&version, sizeof(int), 1, fp);
    fwrite(&r->count, sizeof(int), 1, fp);
    fwrite(&r->nextId, sizeof(int), 1, fp);
    fwrite(stock, sizeof(int), NUM_GROUPS, fp);
    fwrite(r->list, sizeof(Donor), r->count, fp);
    fclose(fp);
    return 1;
}

/*
 * Random access: after a donation or an update only the stock and ONE
 * donor record change, so fseek jumps straight to them in the file
 * instead of rewriting the whole file.
 */
int saveRecord(const Registry *r, int idx)
{
    FILE *fp = fopen(DATA_FILE, "r+b");

    if (fp == NULL)
        return saveData(r);
    fseek(fp, (long)(3 * sizeof(int)), SEEK_SET);                  /* stock */
    fwrite(stock, sizeof(int), NUM_GROUPS, fp);
    fseek(fp, HEADER_SIZE + (long)idx * (long)sizeof(Donor), SEEK_SET);   /* record idx */
    fwrite(&r->list[idx], sizeof(Donor), 1, fp);
    fclose(fp);
    return 1;
}

int loadData(Registry *r)
{
    FILE *fp = fopen(DATA_FILE, "rb");
    int version, count, nextId;

    if (fp == NULL)
        return 0;

    /* the file starts with a version number: a file from an older program is not used */
    if (fread(&version, sizeof(int), 1, fp) != 1 || version != FILE_VERSION ||
        fread(&count, sizeof(int), 1, fp) != 1 ||
        fread(&nextId, sizeof(int), 1, fp) != 1 ||
        fread(stock, sizeof(int), NUM_GROUPS, fp) != NUM_GROUPS ||
        count < 0) {
        printf("  ! %s is from an older version or damaged - starting with sample data.\n", DATA_FILE);
        fclose(fp);
        return 0;
    }

    r->capacity = count > 8 ? count : 8;
    r->list = calloc(r->capacity, sizeof(Donor));   /* zero-filled */
    if (r->list == NULL || (int)fread(r->list, sizeof(Donor), count, fp) != count) {
        free(r->list);
        r->list = NULL;
        r->capacity = 0;
        fclose(fp);
        return 0;
    }
    r->count = count;
    r->nextId = nextId;
    fclose(fp);
    return 1;
}

/* used on the very first run so the program has data to demonstrate */
void loadSampleData(Registry *r)
{
    const Donor samples[] = {
        {0, "Ravi Kumar", "K. Srinivas", {14, 3, 2002}, 'M', "O+", 68, "9876543210", "",
         "12-4 Seed Access Road", "Amaravati", "Guntur", "Andhra Pradesh", "522237", "4821", 0, {12, 5, 2026}, 3},
        {0, "Sneha Reddy", "P. Venkat Reddy", {22, 7, 2005}, 'F', "A+", 54, "9123456780", "",
         "45 Benz Circle", "Vijayawada", "NTR", "Andhra Pradesh", "520010", "3317", 0, {1, 9, 2026}, 1},
        {0, "Arjun Varma", "R. Suresh Varma", {5, 1, 1996}, 'M', "B+", 72, "9988776655", "",
         "7 Brodipet 4th Line", "Guntur", "Guntur", "Andhra Pradesh", "522002", "7065", 0, {0, 0, 0}, 0},
        {0, "Priya Sharma", "Anil Sharma", {30, 9, 1999}, 'F', "O-", 58, "8877665544", "",
         "Flat 3B, Rayapudi", "Amaravati", "Guntur", "Andhra Pradesh", "522237", "5420", 0, {15, 6, 2026}, 2},
        {0, "Mohammed Imran", "Mohammed Rafi", {11, 11, 1990}, 'M', "AB+", 80, "7766554433", "",
         "22 One Town", "Vijayawada", "NTR", "Andhra Pradesh", "520001", "8810", 0, {20, 3, 2026}, 5},
        {0, "Divya Lakshmi", "S. Ramana", {17, 2, 2004}, 'F', "A-", 50, "9012345678", "",
```

```
         "3 Arundelpet", "Guntur", "Guntur", "Andhra Pradesh", "522002", "2294", 0, {10, 10, 2026}, 1},
        {0, "Karthik Naidu", "G. Prasad Naidu", {8, 6, 1997}, 'M', "B-", 75, "8123456789", "",
         "9 Thullur Road", "Amaravati", "Guntur", "Andhra Pradesh", "522237", "6603", 0, {5, 1, 2026}, 4},
        {0, "Anjali Rao", "V. Mohan Rao", {3, 12, 1999}, 'F', "AB-", 52, "6301234567", "",
         "16 Labbipet", "Vijayawada", "NTR", "Andhra Pradesh", "520010", "1187", 0, {0, 0, 0}, 0},
        {0, "Suresh Babu", "N. Chandra Babu", {19, 4, 1981}, 'M', "O+", 77, "9440012345", "",
         "5 Kothapet", "Guntur", "Guntur", "Andhra Pradesh", "522001", "4472", 0, {30, 8, 2026}, 6},
        {0, "Lakshmi Prasanna", "B. Ravi Shankar", {25, 8, 1994}, 'F', "A+", 60, "9550067890", "",
         "28 Mandadam", "Amaravati", "Guntur", "Andhra Pradesh", "522237", "9930", 0, {2, 2, 2026}, 2},
        {0, "Ramesh Gupta", "L. Hari Gupta", {9, 1, 1964}, 'M', "O+", 70, "9000012345", "",
         "11 Lakshmipuram", "Guntur", "Guntur", "Andhra Pradesh", "522007", "5518", 0, {0, 0, 0}, 0}
    };
    const int sampleStock[NUM_GROUPS] = {12, 3, 15, 2, 7, 1, 20, 4};
    int i, n = (int)(sizeof samples / sizeof samples[0]);

    r->count = 0;
    r->nextId = 1;
    for (i = 0; i < n; i++) {
        ensureCapacity(r);
        r->list[r->count] = samples[i];
        r->list[r->count].id = r->nextId++;
        r->list[r->count].aadhaarHash = (unsigned long)(i + 1);   /* sample numbers are not stored */
        r->count++;
    }
    for (i = 0; i < NUM_GROUPS; i++)
        stock[i] = sampleStock[i];
}

/* ---------------------------------------------------------------------
 *  Display helpers
 * --------------------------------------------------------------------- */

void printLine(void)
{
    printf("-----------------------------------------------------------------------------------------------------------------\n");
}

void printHeader(void)
{
    printLine();
    printf("%-4s %-18s %-3s %-3s %-4s %-10s  %-11s %-9s %-12s %-8s %s\n",
           "ID", "Name", "Age", "Sex", "Grp", "Phone", "City", "District", "Last Donated", "Aadhaar", "Status");
    printLine();
}

void statusText(const Donor *d, char *out)
{
    const char *reason;
    int r = checkEligibility(d, &reason);

    if (r == 0)
        strCopy(out, "Eligible");
    else if (r > 0)
        sprintf(out, "Wait %dd", r);
    else
        sprintf(out, "Not eligible (%s)", reason);
}

void printDonor(const Donor *d)
{
    char last[16], status[48];

    formatDate(d->lastDonation, last);
    statusText(d, status);
    printf("%-4d %-18.18s %-3d  %c  %-4s %-10s  %-11.11s %-9.9s %-12s xx%-6s %s\n",
           d->id, d->name, ageOn(d->dob, today), d->gender, d->group, d->phone,
           d->city, d->district, last, d->aadhaarLast4, status);
}

/* full details of one donor, as on a donor card */
void printDonorCard(const Donor *d)
{
    char dob[16], last[16], status[48];

    formatDate(d->dob, dob);
    formatDate(d->lastDonation, last);
    statusText(d, status);
    printLine();
    printf("  Donor ID      : %d\n", d->id);
    printf("  Name          : %s\n", d->name);
    printf("  %-14s: %s\n", d->gender == 'M' ? "S/o" : d->gender == 'F' ? "D/o" : "C/o", d->guardian);
    printf("  Date of birth : %s (age %d)\n", dob, ageOn(d->dob, today));
    printf("  Gender        : %c        Blood group: %s        Weight: %d kg\n", d->gender, d->group, d->weight);
    printf("  Mobile        : %s   Email: %s\n", d->phone, d->email[0] ? d->email : "-");
    printf("  Address       : %s, %s,\n", d->address, d->city);
    printf("                  %s district, %s - %s\n", d->district, d->state, d->pin);
    printf("  Aadhaar       : XXXX XXXX %s (verified)\n", d->aadhaarLast4);
    printf("  Last donation : %s   Donations: %d\n", last, d->totalDonations);
    printf("  Status        : %s\n", status);
    printLine();
}

void printBar(int n, char symbol)
{
    int i;
```

```
    for (i = 0; i < n && i < 40; i++)
        putchar(symbol);
}

int lowStockAlert(void)
{
    int i, alerts = 0;

    for (i = 0; i < NUM_GROUPS; i++) {
        if (stock[i] < LOW_STOCK) {
            if (alerts == 0)
                printf("\n  *** LOW STOCK ALERT ***\n");
            printf("  %-4s only %d unit(s) left - organise a donation drive!\n", GROUPS[i], stock[i]);
            alerts++;
        }
    }
    return alerts;
}

/* ---------------------------------------------------------------
 *  Sorting (the compare rule is passed as a pointer to a function)
 * --------------------------------------------------------------- */

int byName(const Donor *x, const Donor *y)
{
    return compareIgnoreCase(x->name, y->name);
}

int byGroup(const Donor *x, const Donor *y)
{
    int diff = groupIndex(x->group) - groupIndex(y->group);
    return diff ? diff : compareIgnoreCase(x->name, y->name);
}

int byCity(const Donor *x, const Donor *y)
{
    int diff = compareIgnoreCase(x->city, y->city);
    return diff ? diff : compareIgnoreCase(x->name, y->name);
}

/* insertion sort of an array of donor pointers, using any compare function */
void sortDonors(const Donor *view[], int n, int (*compare)(const Donor *, const Donor *))
{
    int i, j;

    for (i = 1; i < n; i++) {
        const Donor *key = view[i];

        for (j = i - 1; j >= 0 && compare(view[j], key) > 0; j--)
            view[j + 1] = view[j];
        view[j + 1] = key;
    }
}

/* ---------------------------------------------------------------
 *  Menu features
 * --------------------------------------------------------------- */

void registerDonor(Registry *r)
{
    Donor d = {0};          /* every member starts at zero */
    char reasons[160];
    int age;

    printf("\n--- Register as a Blood Donor ---\n");
    printf("Rules (NBTC 2017): age 18-65 (first-time donor up to 60), weight 45 kg+,\n");
    printf("90 days between donations for men and 120 days for women.\n");

    printf("\n1. Personal details\n");
    readWords("Full name (as on Aadhaar)  : ", d.name, sizeof d.name);
    readWords("Father's / spouse's name   : ", d.guardian, sizeof d.guardian);
    d.dob = readDate("Date of birth (dd/mm/yyyy) : ", 0);
    age = ageOn(d.dob, today);
    if (age < MIN_AGE || age > MAX_AGE) {
        printf("  ! Age %d: a donor must be %d-%d years old. Registration cancelled.\n", age, MIN_AGE, MAX_AGE);
        return;
    }
    printf("  Age: %d years\n", age);
    d.gender = readGender("Gender (M/F/O)            : ");
    readBloodGroup("Blood group               : ", d.group);
    d.weight = readInt("Body weight (kg)          : ", 1, 250);
    if (d.weight < MIN_WEIGHT) {
        printf("  ! Minimum weight for donation is %d kg. Registration cancelled.\n", MIN_WEIGHT);
        return;
    }
    readPhone("Mobile number             : ", d.phone);
    readEmail("Email (Enter to skip)     : ", d.email, sizeof d.email);

    printf("\n2. Address\n");
    readAddress("House no. / street        : ", d.address, sizeof d.address);
    readWords("City / town               : ", d.city, sizeof d.city);
    readWords("District                  : ", d.district, sizeof d.district);
    readState("State / UT                : ", d.state);
    readPin("PIN code                  : ", d.pin);
```

```
    printf("\n3. Aadhaar verification\n");
    readAadhaar("Aadhaar number             : ", r, d.aadhaarLast4, &d.aadhaarHash);
    if (!readYesNo("I consent to the use of my Aadhaar only to verify my identity\nfor blood donation (Aadhaar Act, 2016).
Agree? (y/n): ")) {
        printf("  ! Consent is required to verify identity. Registration cancelled.\n");
        return;
    }

    printf("\n4. Donation history\n");
    for (;;) {
        d.lastDonation = readDate("Last donation date (dd/mm/yyyy, 0 if never): ", 1);
        if (d.lastDonation.y == 0 || ageOn(d.dob, d.lastDonation) >= MIN_AGE)
            break;
        printf("  ! The donor was under %d on that date.\n", MIN_AGE);
    }
    if (d.lastDonation.y == 0 && age > MAX_FIRST_AGE) {
        printf("  ! A first-time donor must not be above %d (age %d). Registration cancelled.\n", MAX_FIRST_AGE, age);
        return;
    }
    d.totalDonations = d.lastDonation.y ? 1 : 0;

    printf("\n5. Health screening (questions from the e-RaktKosh portal)\n");
    if (screening(1, reasons, sizeof reasons) > 0) {
        printf("  ! Not eligible to register right now: %s.\n", reasons);
        printf("    Please register after the deferral period. Registration cancelled.\n");
        return;
    }

    printf("\n6. Declaration\n");
    if (!readYesNo("I declare the information is true and agree to be contacted\nby blood banks for voluntary donation.
Agree? (y/n): ")) {
        printf("  ! The declaration is required. Registration cancelled.\n");
        return;
    }

    ensureCapacity(r);
    d.id = r->nextId++;
    r->list[r->count++] = d;
    saveData(r);

    printf("\n  Aadhaar verified. Donor registered successfully with ID %d.\n", d.id);
    printDonorCard(&d);
}

void listDonors(const Registry *r)
{
    const Donor **view;
    int (*compare)(const Donor *, const Donor *) = NULL;
    int i, choice;

    if (r->count == 0) {
        printf("\n  No donors registered yet.\n");
        return;
    }

    printf("\nSort by: 1. ID   2. Name   3. Blood group   4. City\n");
    choice = readInt("Choice: ", 1, 4);
    switch (choice) {
    case 2: compare = byName;  break;
    case 3: compare = byGroup; break;
    case 4: compare = byCity;  break;
    }

    /* sort an array of pointers so the original list keeps its id order */
    view = calloc(r->count, sizeof *view);
    if (view == NULL) {
        printf("Out of memory!\n");
        return;
    }
    for (i = 0; i < r->count; i++)
        view[i] = r->list + i;          /* same address as &r->list[i] */
    if (compare)
        sortDonors(view, r->count, compare);

    printf("\n");
    printHeader();
    for (i = 0; i < r->count; i++)
        printDonor(view[i]);
    printLine();
    printf("Total donors: %d\n", r->count);
    free(view);
}

int matchesKey(const Donor *d, int mode, const SearchKey *key)
{
    switch (mode) {
    case BY_GROUP:    return strCompare(d->group, key->group) == 0;
    case BY_CITY:     return compareIgnoreCase(d->city, key->place) == 0;
    case BY_DISTRICT: return compareIgnoreCase(d->district, key->place) == 0;
    default:          return d->id == key->id;
    }
}

void searchDonors(const Registry *r)
```

```
{
    SearchKey key;
    const Donor *d;
    int mode, found = 0, eligible = 0;

    printf("\n--- Search Donors ---\n");
    printf("Search by: 1. Blood group   2. City   3. District   4. Donor ID\n");
    mode = readInt("Choice: ", 1, 4);
    switch (mode) {
    case BY_GROUP:    readBloodGroup("Blood group : ", key.group);               break;
    case BY_CITY:     readWords("City        : ", key.place, sizeof key.place);  break;
    case BY_DISTRICT: readWords("District    : ", key.place, sizeof key.place);  break;
    default:          key.id = readInt("Donor ID    : ", 1, 999999);            break;
    }

    printf("\n");
    /* pointer arithmetic: d walks from the first donor to the last */
    for (d = r->list; d < r->list + r->count; d++) {
        if (!matchesKey(d, mode, &key))
            continue;
        if (mode == BY_ID) {          /* one donor: show the full details */
            printDonorCard(d);
            return;
        }
        if (found == 0)
            printHeader();
        printDonor(d);
        found++;
        if (isEligible(d))
            eligible++;
    }

    if (found == 0) {
        printf("  No matching donor found.\n");
        return;
    }
    printLine();
    printf("Found %d donor(s), %d eligible to donate now.\n", found, eligible);
}

void emergencyFinder(const Registry *r)
{
    char patient[4], city[30];
    int i, g, pass, found = 0, units = 0, soonestWait = 0;
    const Donor *soonest = NULL;
    const char *reason;

    printf("\n--- EMERGENCY: Find Compatible Donors ---\n");
    readBloodGroup("Patient blood group   : ", patient);
    readLine("City (Enter for all)  : ", city, sizeof city);

    printf("\nPatient %s can receive from: ", patient);
    for (g = 0; g < NUM_GROUPS; g++) {
        if (canDonate(GROUPS[g], patient)) {
            printf("%s ", GROUPS[g]);
            units += stock[g];
        }
    }
    printf("\nCompatible units in blood bank right now: %d\n\n", units);

    /* pass 0: exact blood-group match first, pass 1: other compatible groups */
    for (pass = 0; pass < 2; pass++) {
        for (i = 0; i < r->count; i++) {
            const Donor *d = &r->list[i];
            int exact = strCompare(d->group, patient) == 0;
            int wait;

            if (!canDonate(d->group, patient))
                continue;
            if (city[0] && compareIgnoreCase(d->city, city) != 0)
                continue;
            if ((pass == 0) != exact)
                continue;

            wait = checkEligibility(d, &reason);
            if (wait > 0 && (soonest == NULL || wait < soonestWait)) {
                soonest = d;
                soonestWait = wait;
            }
            if (wait != 0)
                continue;
            if (found == 0)
                printHeader();
            printDonor(d);
            found++;
        }
    }

    if (found > 0) {
        printLine();
        printf("%d eligible donor(s) found. Exact matches are listed first - call them now.\n", found);
    } else {
        printf("  No eligible compatible donor available right now.\n");
        if (soonest)
            printf("  Next available: %s (%s, %s) in %d day(s), phone %s.\n",
```

```
                    soonest->name, soonest->group, soonest->city, soonestWait, soonest->phone);
    }
}

void recordDonation(Registry *r)
{
    Donor *d;
    int idx, wait, g;
    const char *reason;
    char reasons[160];

    printf("\n--- Record a Donation ---\n");
    idx = askForDonor(r);
    if (idx < 0)
        return;
    d = &r->list[idx];

    printHeader();
    printDonor(d);
    printLine();

    wait = checkEligibility(d, &reason);
    if (wait > 0) {
        printf("  ! %s must wait %d more day(s): NBTC requires a %d-day gap for %s.\n",
               d->name, wait, gapDays(d->gender), genderWord(d->gender));
        printf("    Donation not recorded.\n");
        return;
    }
    if (wait < 0) {
        printf("  ! %s is not eligible (%s). Donation not recorded.\n", d->name, reason);
        return;
    }

    printf("Pre-donation screening (answer for today):\n");
    if (screening(0, reasons, sizeof reasons) > 0) {
        printf("  ! %s is deferred today (%s). Donation not recorded.\n", d->name, reasons);
        return;
    }
    if (!readYesNo("Record one unit donated today? (y/n): "))
        return;

    d->lastDonation = today;
    d->totalDonations++;
    g = groupIndex(d->group);
    stock[g]++;
    saveRecord(r, idx);

    printf("\n  Thank you, %s! Donation #%d recorded.\n", d->name, d->totalDonations);
    printf("  %s stock is now %d unit(s). Next donation allowed after %d days.\n",
           d->group, stock[g], gapDays(d->gender));
}

void updateDonor(Registry *r)
{
    Donor *d;
    int idx, choice;

    printf("\n--- Update Donor Details ---\n");
    idx = askForDonor(r);
    if (idx < 0)
        return;
    d = &r->list[idx];
    printDonorCard(d);

    printf("1. Mobile   2. Email   3. Address   4. Weight   5. Cancel\n");
    choice = readInt("Choice: ", 1, 5);
    switch (choice) {
    case 1: readPhone("New mobile number : ", d->phone);                    break;
    case 2: readEmail("New email         : ", d->email, sizeof d->email);   break;
    case 3:
        readAddress("House no. / street: ", d->address, sizeof d->address);
        readWords("City / town       : ", d->city, sizeof d->city);
        readWords("District          : ", d->district, sizeof d->district);
        readState("State / UT        : ", d->state);
        readPin("PIN code          : ", d->pin);
        break;
    case 4: d->weight = readInt("Body weight (kg)  : ", 1, 250);            break;
    default: return;
    }
    saveRecord(r, idx);
    printf("  Donor %d updated.\n", d->id);
}

void deleteDonor(Registry *r)
{
    int idx, i;

    printf("\n--- Remove Donor ---\n");
    idx = askForDonor(r);
    if (idx < 0)
        return;

    printHeader();
    printDonor(&r->list[idx]);
    printLine();
```

```
    if (!readYesNo("Remove this donor permanently? (y/n): "))
        return;

    /* shift the remaining donors left; order by id is preserved */
    for (i = idx; i < r->count - 1; i++)
        r->list[i] = r->list[i + 1];
    r->count--;
    saveData(r);
    printf("  Donor removed.\n");
}

void viewStock(void)
{
    int g, total = 0;

    printf("\n--- Blood Stock ---\n");
    for (g = 0; g < NUM_GROUPS; g++) {
        printf("  %-4s %3d unit(s) |", GROUPS[g], stock[g]);
        printBar(stock[g], '#');
        if (stock[g] < LOW_STOCK)
            printf("  <-- LOW");
        printf("\n");
        total += stock[g];
    }
    printf("  Total: %d unit(s)\n", total);
    lowStockAlert();
}

void issueBlood(Registry *r)
{
    char patient[4], other[4];
    int units, g, alt, anyAlt = 0;

    printf("\n--- Issue Blood to a Patient ---\n");
    readBloodGroup("Patient blood group : ", patient);
    units = readInt("Units required (1-10): ", 1, 10);
    g = groupIndex(patient);

    if (stock[g] >= units) {
        stock[g] -= units;
        saveData(r);
        printf("  Issued %d unit(s) of %s. Remaining %s stock: %d.\n", units, patient, patient, stock[g]);
        lowStockAlert();
        return;
    }

    printf("  ! Only %d unit(s) of %s available.\n", stock[g], patient);
    printf("  Compatible groups with enough stock: ");
    for (alt = 0; alt < NUM_GROUPS; alt++) {
        if (alt != g && canDonate(GROUPS[alt], patient) && stock[alt] >= units) {
            printf("%s(%d) ", GROUPS[alt], stock[alt]);
            anyAlt = 1;
        }
    }
    if (!anyAlt) {
        printf("none.\n  Use option 4 (Emergency) to call eligible donors.\n");
        return;
    }
    printf("\n");
    if (!readYesNo("Issue from a compatible group instead? (y/n): "))
        return;

    readBloodGroup("Group to issue from : ", other);
    alt = groupIndex(other);
    if (!canDonate(other, patient)) {
        printf("  ! %s is NOT compatible with %s. Not issued.\n", other, patient);
        return;
    }
    if (stock[alt] < units) {
        printf("  ! Not enough %s stock. Not issued.\n", other);
        return;
    }
    stock[alt] -= units;
    saveData(r);
    printf("  Issued %d unit(s) of %s for a %s patient. Remaining %s stock: %d.\n",
           units, other, patient, other, stock[alt]);
    lowStockAlert();
}

void stockMenu(Registry *r)
{
    printf("\n1. View stock   2. Issue blood   3. Back\n");
    switch (readInt("Choice: ", 1, 3)) {
    case 1: viewStock();   break;
    case 2: issueBlood(r); break;
    default: break;
    }
}

void statistics(const Registry *r)
{
    int perGroup[NUM_GROUPS] = {0};
    int i, g, eligible = 0, donations = 0, top = 0;

    for (i = 0; i < r->count; i++) {
```

```
        perGroup[groupIndex(r->list[i].group)]++;
        donations += r->list[i].totalDonations;
        if (isEligible(&r->list[i]))
            eligible++;
    }
    for (g = 1; g < NUM_GROUPS; g++)
        if (perGroup[g] > perGroup[top])
            top = g;

    printf("\n--- Statistics ---\n");
    printf("  Registered donors      : %d\n", r->count);
    printf("  Eligible to donate now : %d\n", eligible);
    printf("  Donations recorded     : %d\n", donations);
    if (r->count > 0)
        printf("  Most common group      : %s\n", GROUPS[top]);

    printf("\n  Donors per blood group:\n");
    for (g = 0; g < NUM_GROUPS; g++) {
        printf("  %-4s %2d |", GROUPS[g], perGroup[g]);
        printBar(perGroup[g] * 2, '*');
        printf("\n");
    }
}

/* writes a readable text report with fprintf, e.g. to print or share */
void exportReport(const Registry *r)
{
    FILE *fp = fopen(REPORT_FILE, "w");
    const Donor *d;
    char date[16], last[16], status[48];
    int g, eligible = 0;

    if (fp == NULL) {
        printf("  ! Could not create %s\n", REPORT_FILE);
        return;
    }
    formatDate(today, date);

    fprintf(fp, "LifeLink - Blood Bank & Donor Finder\n");
    fprintf(fp, "Donor report generated on %s\n", date);
    fprintf(fp, "Eligibility: NBTC 2017 - age 18-65 (first-time up to 60), weight 45 kg+,\n");
    fprintf(fp, "             90 days (men) / 120 days (women) between donations\n\n");
    fprintf(fp, "%-4s %-18s %-3s %-3s %-4s %-10s  %-11s %-9s %-6s %-12s %-8s %s\n",
            "ID", "Name", "Age", "Sex", "Grp", "Phone", "City", "District", "PIN", "Last Donated", "Aadhaar", "Status");
    for (d = r->list; d < r->list + r->count; d++) {
        formatDate(d->lastDonation, last);
        statusText(d, status);
        if (isEligible(d))
            eligible++;
        fprintf(fp, "%-4d %-18.18s %-3d  %c  %-4s %-10s  %-11.11s %-9.9s %-6s %-12s xx%-6s %s\n",
                d->id, d->name, ageOn(d->dob, today), d->gender, d->group, d->phone,
                d->city, d->district, d->pin, last, d->aadhaarLast4, status);
    }

    fprintf(fp, "\nBlood stock (units):\n");
    for (g = 0; g < NUM_GROUPS; g++)
        fprintf(fp, "  %-4s %3d%s\n", GROUPS[g], stock[g], stock[g] < LOW_STOCK ? "  LOW" : "");
    fprintf(fp, "\nTotal donors: %d   Eligible now: %d\n", r->count, eligible);
    fclose(fp);

    printf("\n  Report of %d donors written to %s\n", r->count, REPORT_FILE);
}

/* ------------------------------------------------------------------
 *  Main
 * ------------------------------------------------------------------ */

void printMenu(const Registry *r)
{
    char date[16];
    int g, total = 0;

    for (g = 0; g < NUM_GROUPS; g++)
        total += stock[g];
    formatDate(today, date);

    printf("\n================================================\n");
    printf("      LifeLink - Blood Bank & Donor Finder\n");
    printf("================================================\n");
    printf(" Today: %s   Donors: %d   Stock: %d units\n", date, r->count, total);
    printf("------------------------------------------------\n");
    printf(" 1. Register new donor\n");
    printf(" 2. View all donors\n");
    printf(" 3. Search donors (group / city / district / ID)\n");
    printf(" 4. EMERGENCY - find compatible donors\n");
    printf(" 5. Record a donation\n");
    printf(" 6. Update donor details\n");
    printf(" 7. Remove donor\n");
    printf(" 8. Blood stock (view / issue)\n");
    printf(" 9. Statistics\n");
    printf("10. Export donor report (text file)\n");
    printf(" 0. Save and exit\n");
    printf("------------------------------------------------\n");
}
```

```
int main(void)
{
    Registry reg = {NULL, 0, 0, 1};
    int choice;

    printf("LifeLink - Blood Bank & Donor Finder\n");
    today = readToday();
    if (!loadData(&reg)) {
        loadSampleData(&reg);
        saveData(&reg);
        printf("First run: loaded %d sample donors into %s.\n", reg.count, DATA_FILE);
    }
    lowStockAlert();

    do {
        printMenu(&reg);
        choice = readInt("Enter your choice: ", 0, 10);
        switch (choice) {
        case 1: registerDonor(&reg);   break;
        case 2: listDonors(&reg);      break;
        case 3: searchDonors(&reg);    break;
        case 4: emergencyFinder(&reg); break;
        case 5: recordDonation(&reg);  break;
        case 6: updateDonor(&reg);     break;
        case 7: deleteDonor(&reg);     break;
        case 8: stockMenu(&reg);       break;
        case 9: statistics(&reg);      break;
        case 10: exportReport(&reg);   break;
        case 0:
            if (saveData(&reg))
                printf("\nData saved to %s. Stay healthy - donate blood, save lives!\n", DATA_FILE);
            break;
        }
    } while (choice != 0);

    free(reg.list);
    return 0;
}
```

Output screenshots

Captured from actual runs in the terminal. The first 8 digits of the test Aadhaar numbers are masked for privacy.

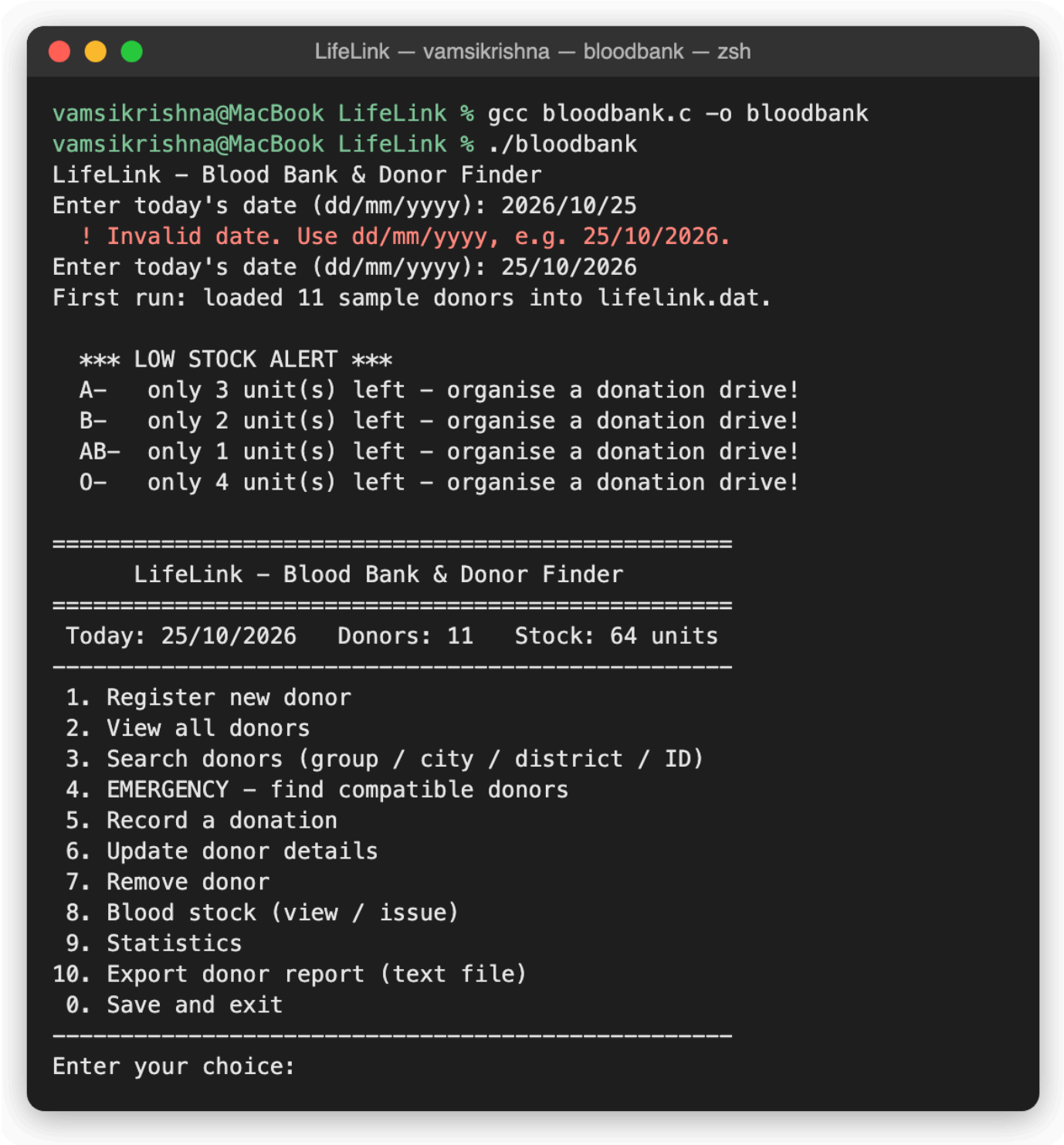


Fig. 1. Compiling and the first run: today's date is entered (a wrong format is rejected), sample donors load, low-stock alert and the main menu.

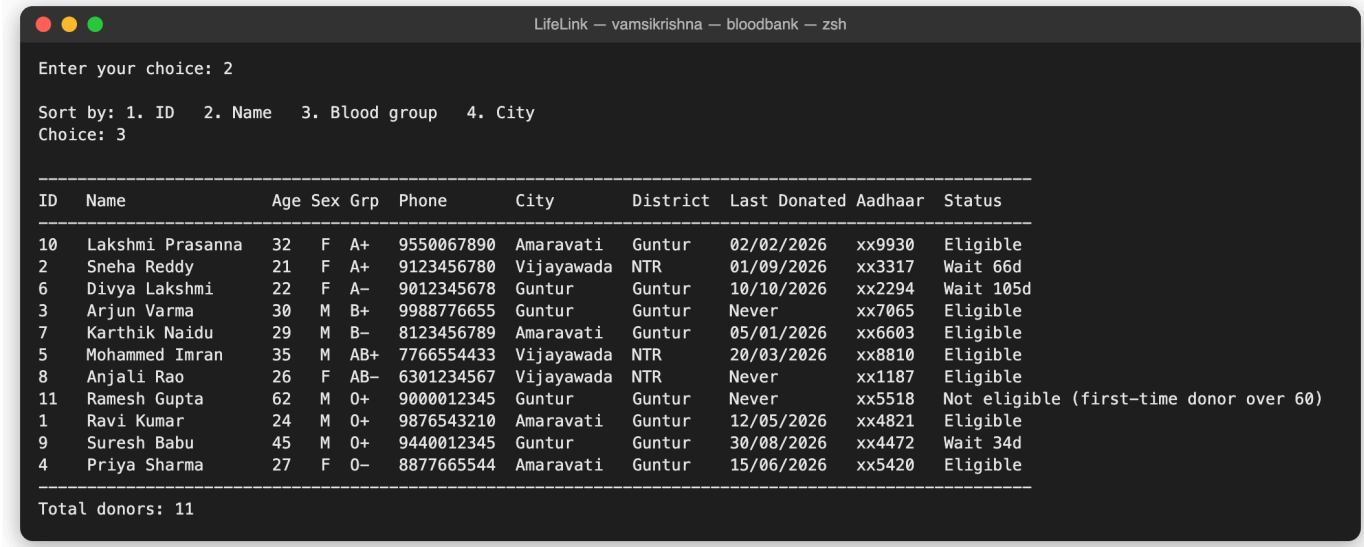


Fig. 2. All donors sorted by blood group, with age, district, masked Aadhaar and NBTC eligibility status.



Fig. 3. Registering a donor in six steps; wrong blood group, mobile, email, state, PIN and Aadhaar checksum are rejected.



Fig. 4. Registrations refused: a duplicate Aadhaar, a screening deferral (recent tattoo) and a donor under 45 kg.

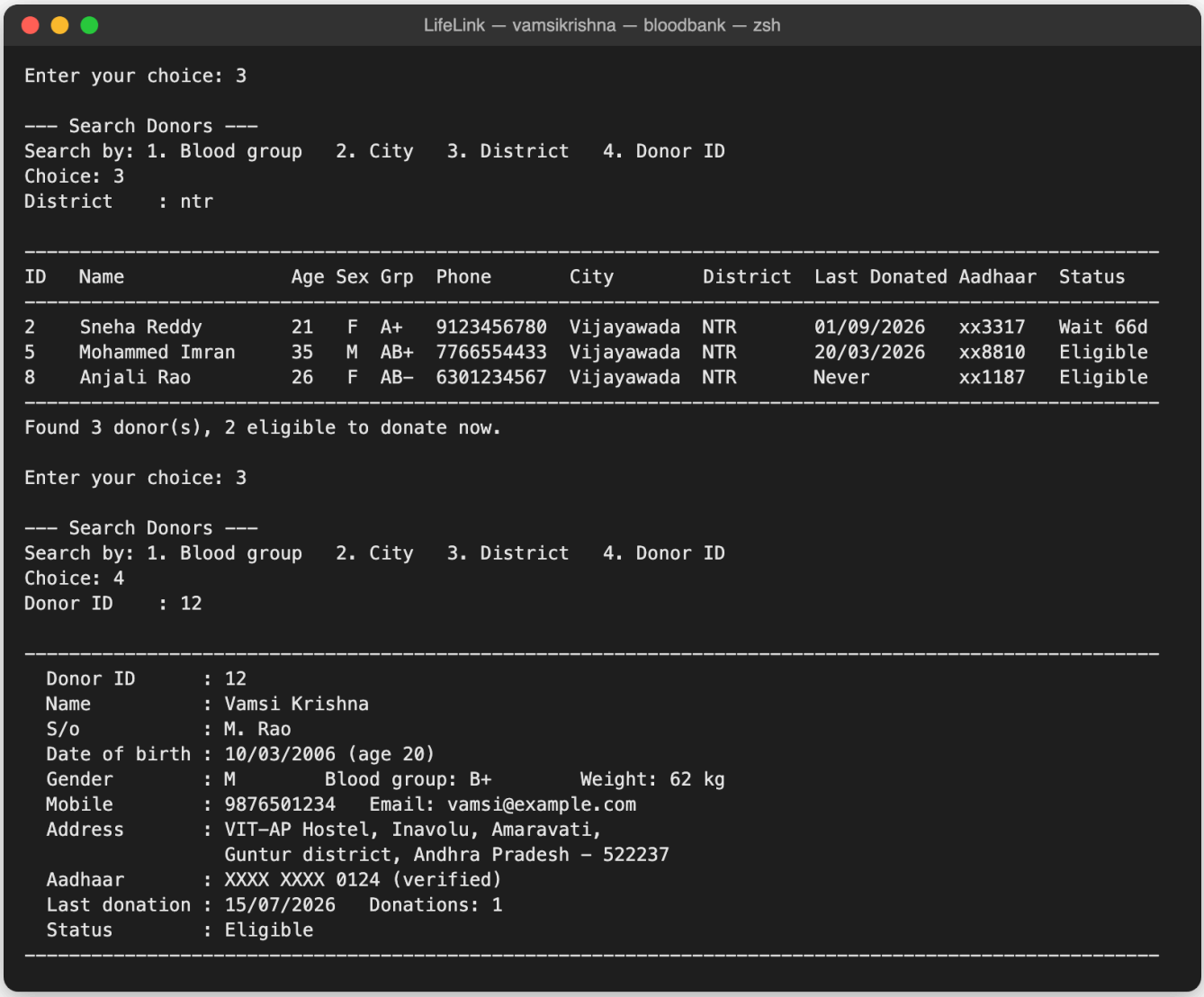


Fig. 5. Search by district (union search key) and by donor ID, which shows the full donor card.

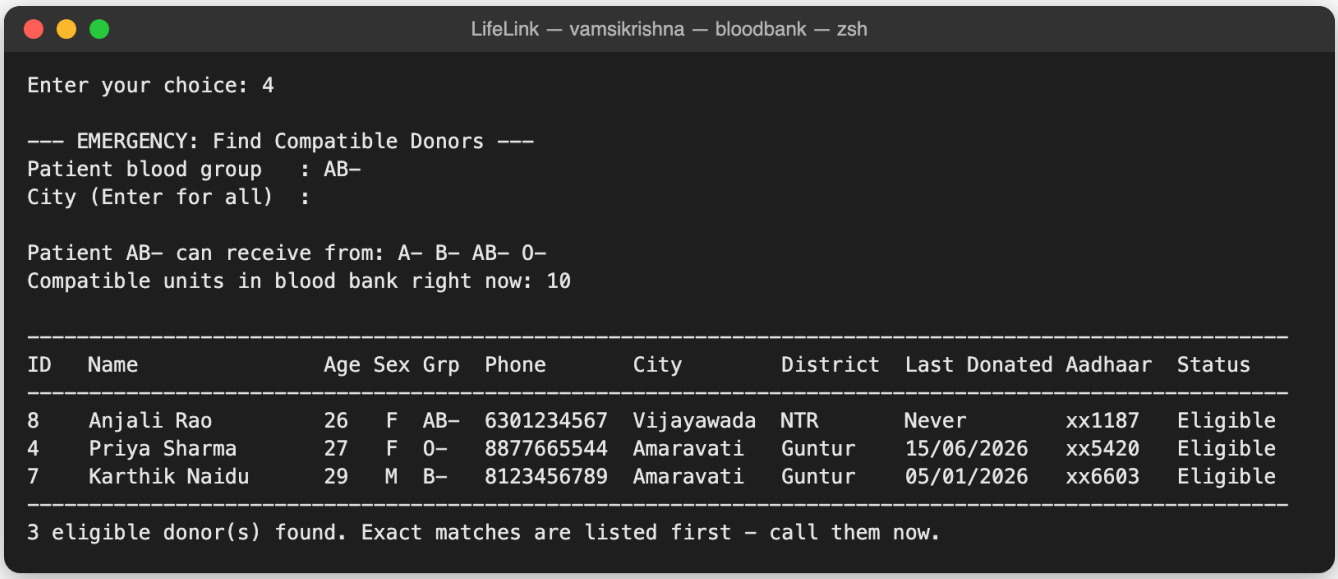


Fig. 6. Emergency finder for an AB- patient: compatible groups, units in stock and eligible donors, exact match first.

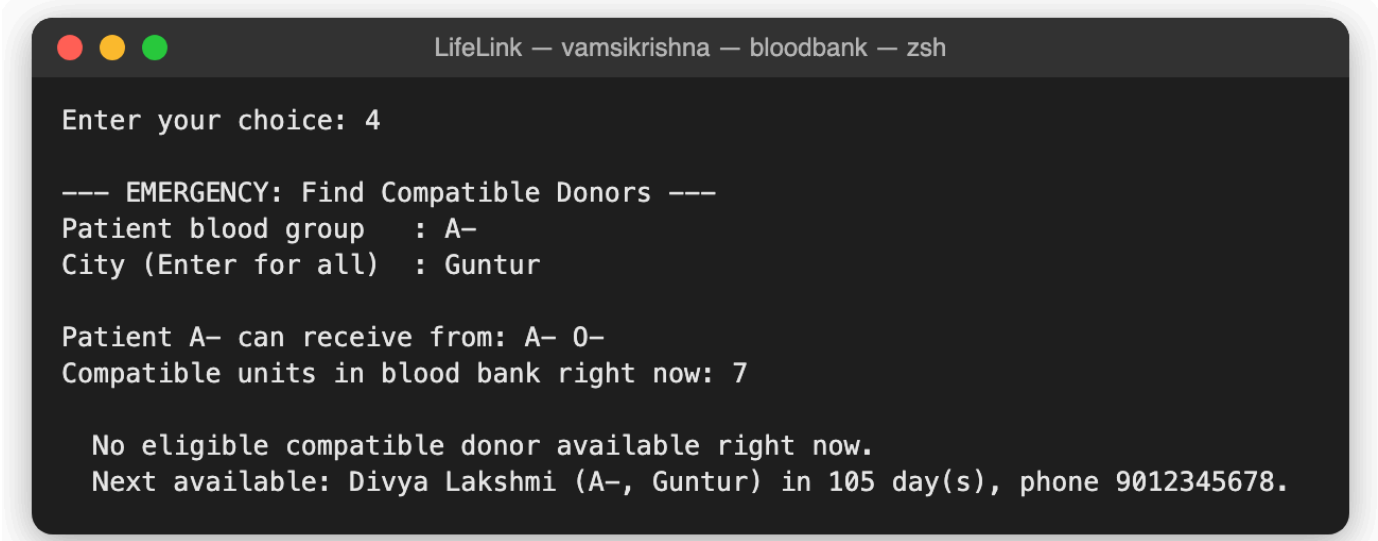


Fig. 7. No eligible donor: the donor available soonest is shown (120-day rule for women).

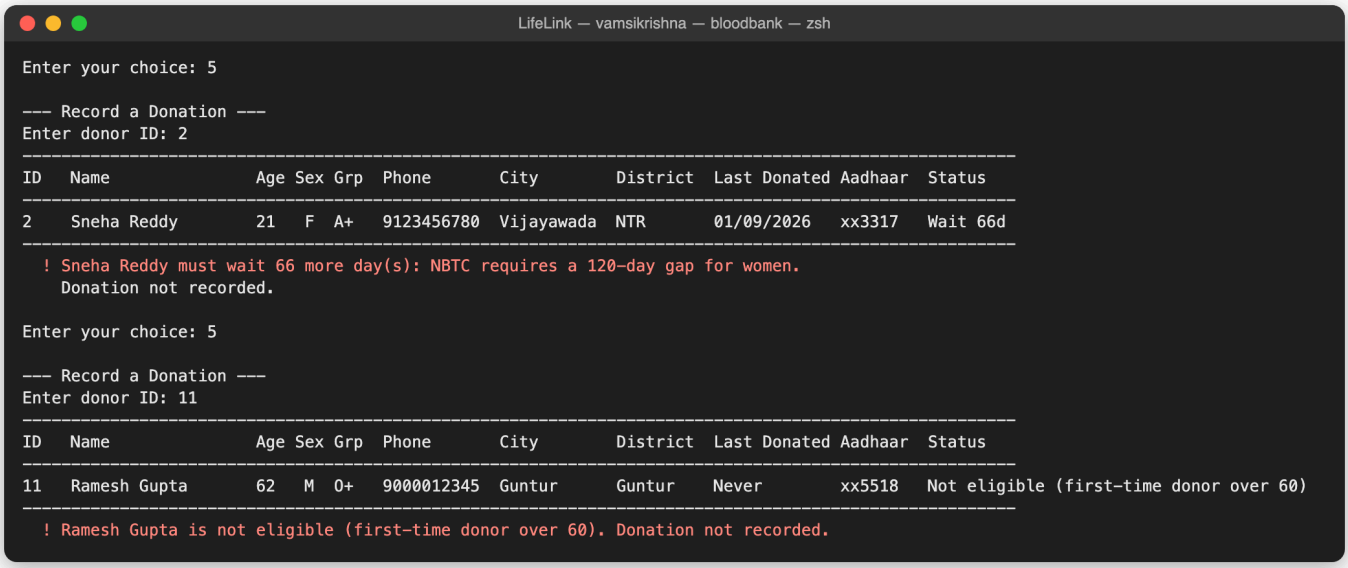


Fig. 8. Donations refused by NBTC rules: a woman inside the 120-day gap and a first-time donor aged 62.

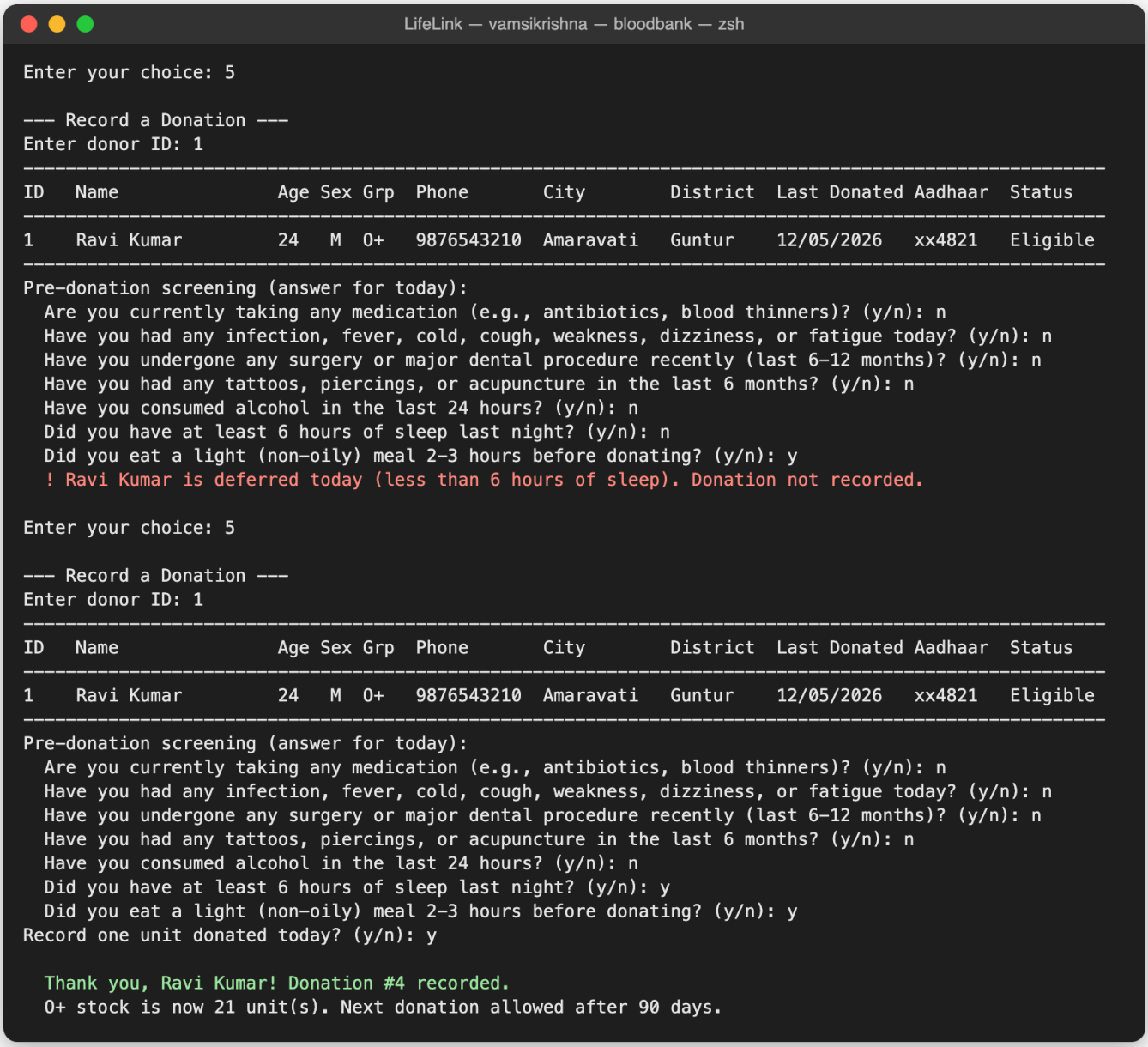


Fig. 9. Pre-donation screening: deferred for less than 6 hours of sleep, then accepted and recorded.

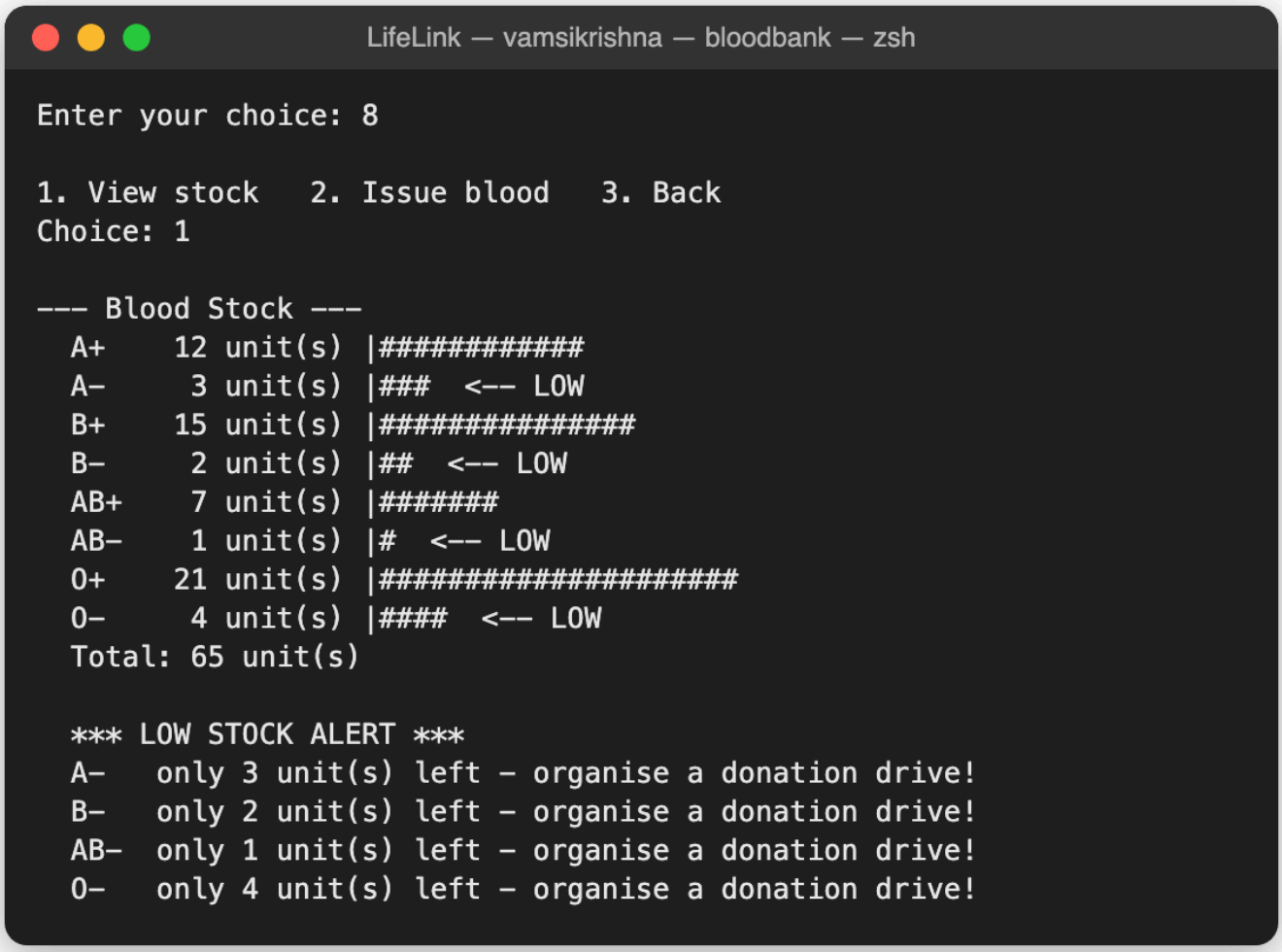


Fig. 10. Blood stock as a bar chart with LOW markers.

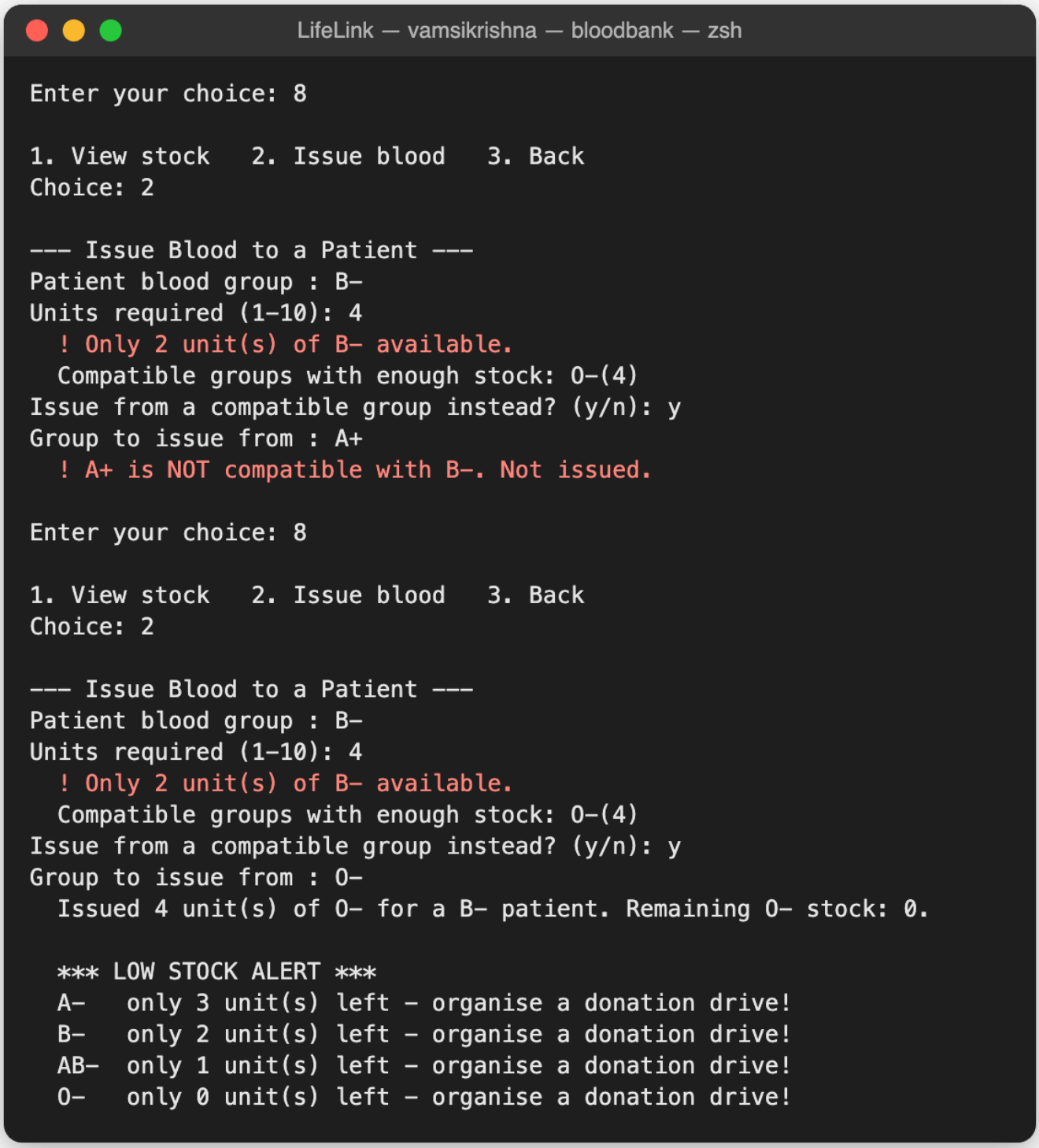


Fig. 11. Issuing blood: an incompatible substitute (A+) is blocked; a compatible one (O-) is issued.

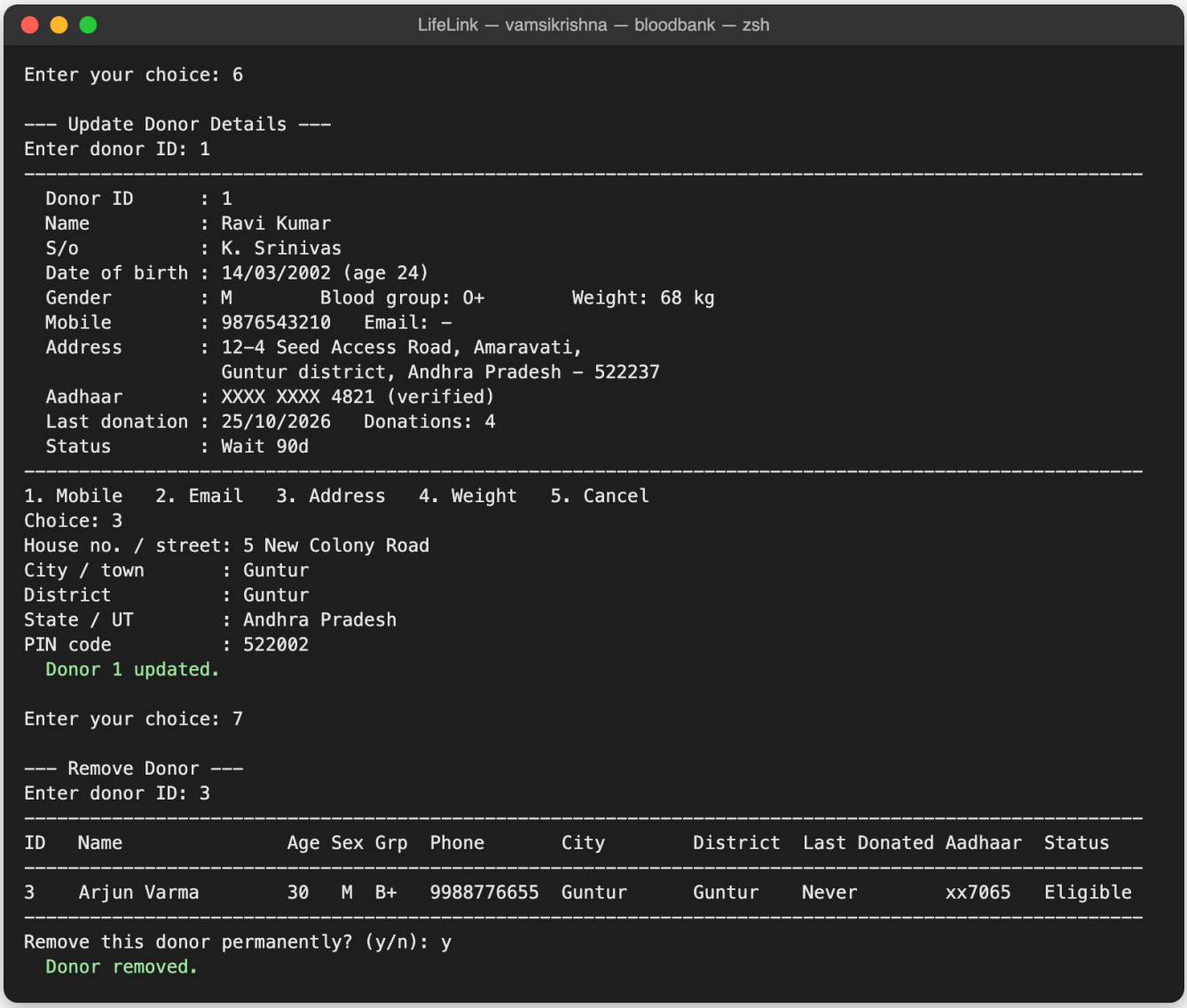


Fig. 12. Updating a donor's address (saved with fseek) and removing a donor.

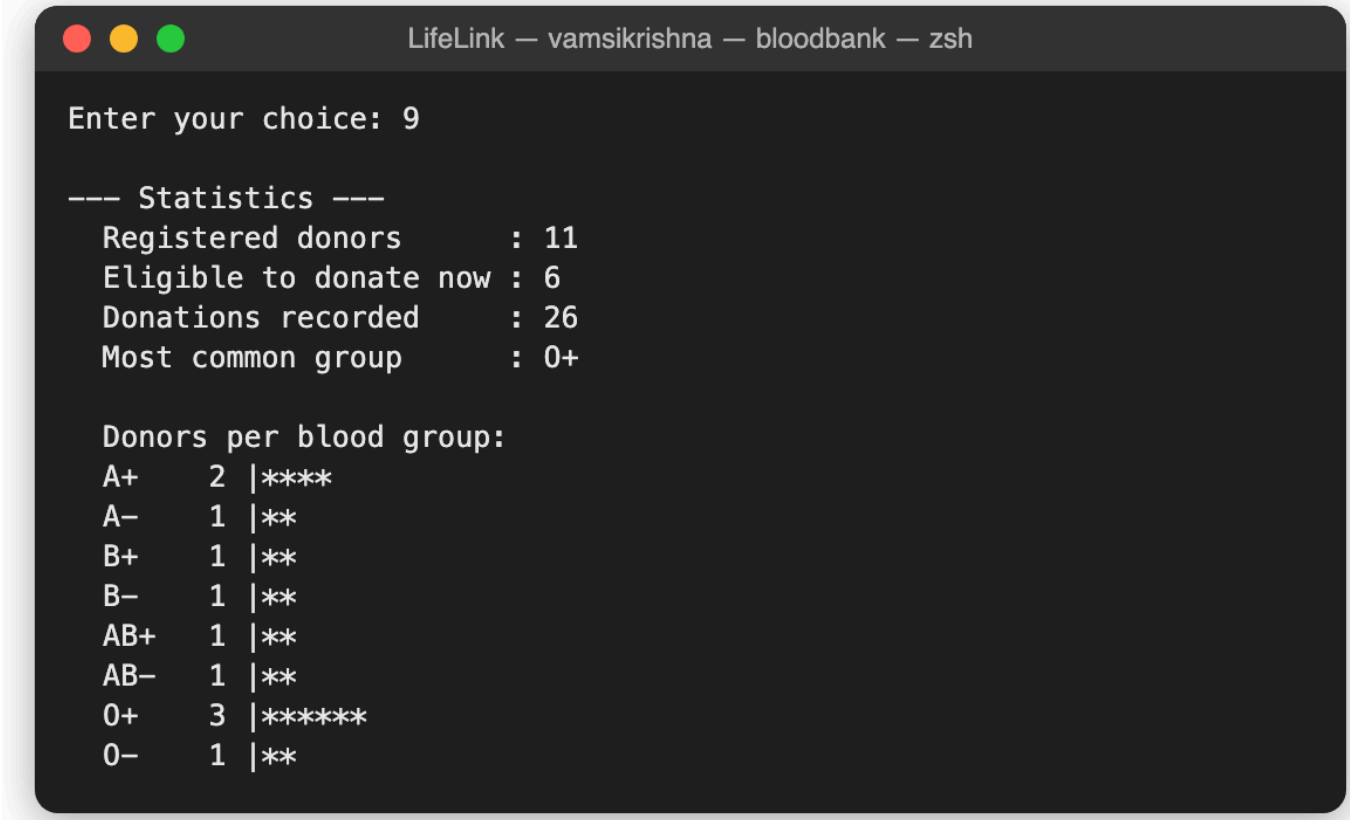


Fig. 13. Statistics with a donors-per-group chart.

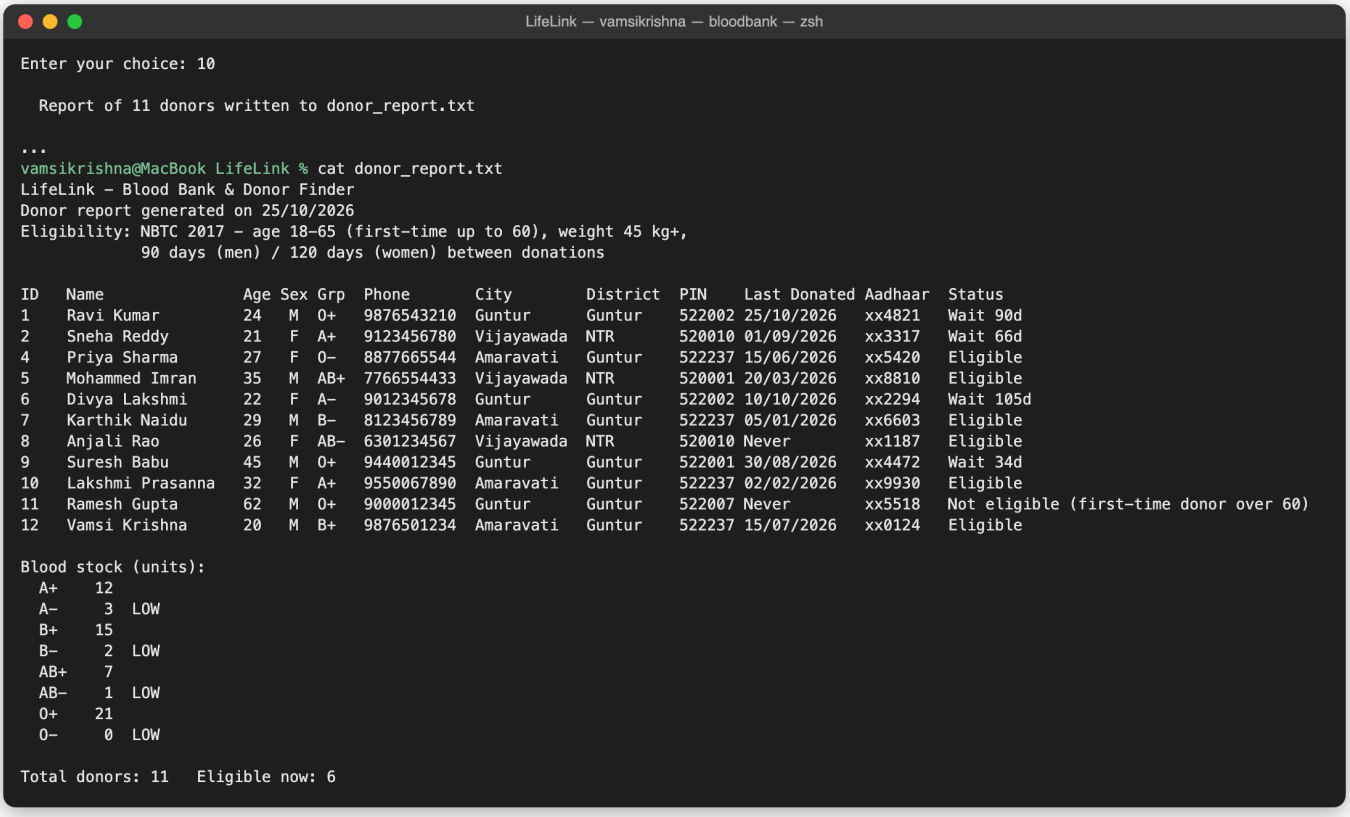


Fig. 14. The text report written with fprintf, shown with cat.

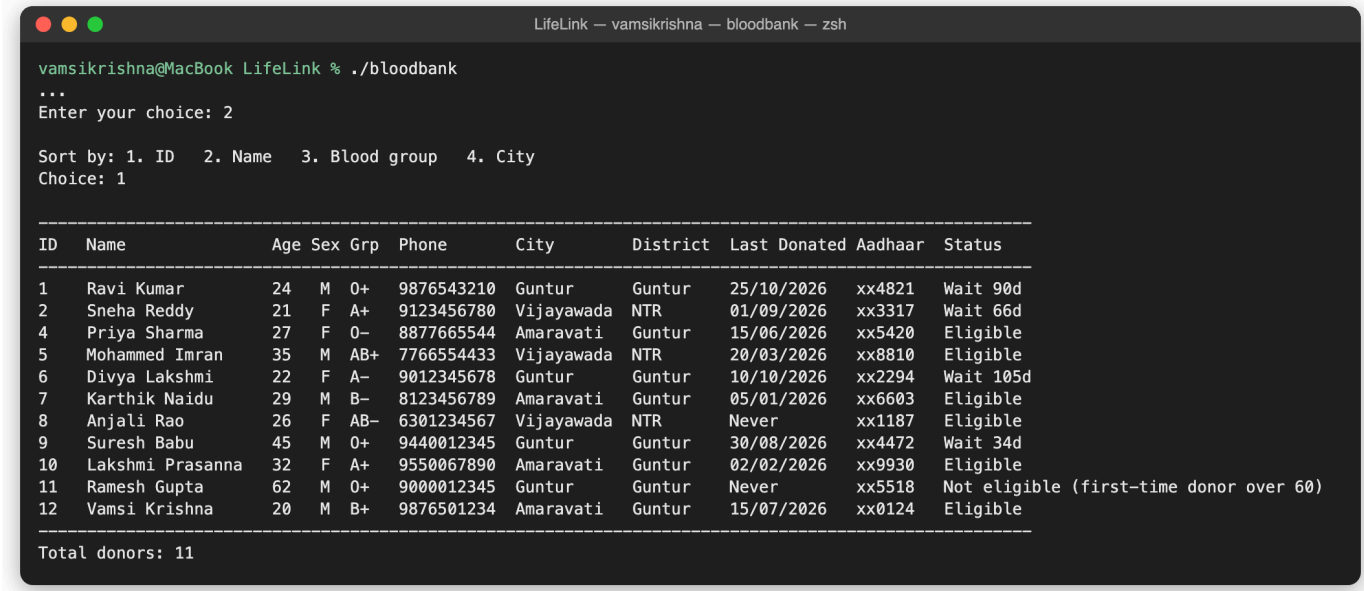


Fig. 15. After a restart, every change is still there (binary file).